Werkzeuge · Warm-up-Karten

Warm-up-Karten Welt 2 – Die Rangliste

Kurze Papier-Aufgaben für den Einstieg jeder Doppelstunde („10' Papier-Warm-up, ohne Gerät"). Jede Karte ist in 5–8 Minuten handschriftlich lösbar und trainiert genau die Formate, die im Abitur zu Suchen und Sortieren verlangt werden – Tracetabellen zu Selection-, Insertion- und Bubblesort und zur binären Suche, Struktogramme, Vergleiche zählen und Java von Hand.

So funktioniert's

- Karte ziehen (oder zugeteilt bekommen), **ohne Gerät** auf einem eigenen Blatt lösen.
- Selbst- oder Partnerkontrolle mit dem Lösungsteil (gleiche Nummer). Papier-Warm-up gibt keine Punkte – sie trainieren die Handschrift fürs Abitur.
- Die gleichen Karten gibt es digital im **Warm-up-Trainer** des Algo Quest Lab (algoquest.frankjuchim.de, Welt 2): Lernkartei nach Leitner mit Selbsteinschätzung, ohne Punkte. Boss-Training vor Boss 2: Karten W2-06, W2-09, W2-12 und W2-14.
- Punkte gibt es nur automatisch im Lab (Lab-Level 2.1 bis 2.4, Trainingsplatz, Online-Boss „Das Wettfischen am Drachensee").

Achtung Abi:

Im Abitur schreibst du alles von Hand. Achte auf die Abi-Check-Liste: Signatur wie gefordert · Rückgabe in jedem Pfad · Leer-/Index-/null-Abfrage vor jedem Zugriff · Schleifengrenzen 0 bis anzahl − 1 · Zeichenketten mit equals vergleichen. Effizienz wird im Abitur über Zählen (Vergleiche, Durchläufe), Worst-Case-Eingaben und Verdopplung geprüft – nicht über O-Notation.

Kartenübersicht

Nr.	Kategorie	Titel	Min.	Einsatz
W2-01	Fehler im Java-Code	Wie oft hat Aria gespielt?	6	DS 13
W2-02	Fehler im Java-Code	Die Minusrunde	7	DS 13
W2-03	Methode implementieren	Die Späher der Rangliste	7	DS 13
W2-04	Struktogramm lesen	Das Doppelgänger-Rätsel	6	DS 13
W2-05	Tracetabelle	Selectionsort: Die Auswahl der Besten	6	DS 14
W2-06	Tracetabelle	Insertionsort: Einsortieren wie beim Kartenspiel	7	DS 14 + Boss
W2-07	Lückencode	Die Innenschleife	5	DS 14
W2-08	Tracetabelle	Bubblesort mit Notbremse	6	DS 15
W2-09	Methode implementieren	Neuer Highscore!	8	DS 14 + Boss
W2-10	Verändern und Erweitern	Andersherum und mitgezählt	6	DS 15
W2-11	Code ↔ Struktogramm	Halbieren statt Durchsuchen	6	DS 16
W2-12	Fehler im Struktogramm	Die wackelige Blase	6	DS 15 + Boss
W2-13	Funktionsweise erläutern	Wie lange dauert das Ranking?	7	DS 15
W2-14	Tracetabelle	Die Halbierungs-Jagd	7	DS 16 + Boss

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht.

WARM-UP W2-01 Fehler im Java-Code

Welt 2 – Rangliste · ⏱ 6 min

Wie oft hat Aria gespielt?

Die Methode anzahlSpiele(name: Zeichenkette): Ganzzahl der Klasse Rangliste soll zählen, wie viele Einträge den übergebenen Namen tragen. Sie enthält drei Fehler.

a) Finde die Fehler und erläutere jeweils ihre Auswirkung.

b) Gib eine korrigierte Fassung an.

```
public int anzahlSpiele(String name) {
    int zaehler = 0;
    for (int i = 1; i <= anzahl; i++) {
        if (eintraege[i].getName() == name) {
            zaehler++;
        }
    }
    return zaehler;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-01

WARM-UP W2-02 Fehler im Java-Code

Welt 2 – Rangliste · ⏱ 7 min

Die Minusrunde

Im Hardcore-Modus gibt es Strafpunkte – punkte kann deshalb auch negativ sein.

Die Methode besterEintrag(): Eintrag soll den Eintrag mit den meisten Punkten liefern, bei Gleichstand den ersten, bei leerer Liste null. Sie enthält drei Fehler.

a) Finde die Fehler und gib jeweils eine Rangliste an, bei der sich der Fehler auswirkt.

b) Gib eine korrigierte Fassung an. Denk an die Leer-Abfrage!

```
public Eintrag besterEintrag() {
    int max = 0;
    Eintrag bester = null;
    for (int i = 0; i <= anzahl; i++) {
        if (eintraege[i].getPunkte() >= max) {
            max = eintraege[i].getPunkte();
            bester = eintraege[i];
        }
    }
    return bester;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-02

WARM-UP W2-03 Methode implementieren

Welt 2 – Rangliste · ⏱ 7 min

Die Späher der Rangliste

Implementiere in der Klasse Rangliste (Attribute eintraege: Reihung von Eintrag, anzahl: Ganzzahl):

a) anzahlUeber(grenze: Ganzzahl): Ganzzahl – liefert die Anzahl der Einträge mit mehr als grenze Punkten.

b) indexVon(name: Zeichenkette): Ganzzahl – liefert den Index des ersten Eintrags mit diesem Namen, sonst −1.

Beispiel: Aria 1200, Bruno 900, Cem 1500, Aria 700 → anzahlUeber(900) liefert 2, indexVon("Aria") liefert 0, indexVon("Dana") liefert −1.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-03

WARM-UP W2-04 Struktogramm lesen

Welt 2 – Rangliste · ⏱ 6 min

Das Doppelgänger-Rätsel

Die Operation geheim der Klasse Rangliste ist als Struktogramm gegeben.

a) Die Liste enthält Aria, Bruno, Cem, Bruno (Indizes 0 bis 3). Gib den Rückgabewert an und notiere alle Paare (i, j), für die ein Namensvergleich ausgeführt wird.

b) Beschreibe in einem Satz, was die Operation allgemein prüft.

c) Gib an, wie viele Namensvergleiche bei 10 Einträgen ohne doppelte Namen ausgeführt werden, und begründe.

```
geheim(): Wahrheitswert
  für i von 0 bis anzahl – 2
    für j von i + 1 bis anzahl – 1
      falls eintraege[i].getName() = eintraege[j].getName()
        dann: gib wahr zurück
        sonst: –
  gib falsch zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-04

WARM-UP W2-05 Tracetabelle

Welt 2 – Rangliste · ⏱ 6 min

Selectionsort: Die Auswahl der Besten

Die Methode sortiert eine Reihung von Punkteständen absteigend.

a) Erstelle eine Tracetabelle für r = [40, 70, 20, 90, 50] mit den Spalten i | maxIndex | Reihung nach dem Durchlauf (eine Zeile pro Durchlauf der äußeren Schleife).

b) Gib die Anzahl der Vergleiche r[j] > r[maxIndex] und die Anzahl der Vertauschungen an.

c) Gib an, wie viele Vergleiche ausgeführt werden, wenn r schon absteigend sortiert ist. Begründe.

```
public static void sortiere(int[] r) {
    for (int i = 0; i < r.length - 1; i++) {
        int maxIndex = i;
        for (int j = i + 1; j < r.length; j++) {
            if (r[j] > r[maxIndex]) {
                maxIndex = j;
            }
        }
        if (maxIndex != i) {
            int hilf = r[i];
            r[i] = r[maxIndex];
            r[maxIndex] = hilf;
        }
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-05

WARM-UP W2-06 Tracetabelle

Welt 2 – Rangliste · ⏱ 7 min

Insertionsort: Einsortieren wie beim Kartenspiel

Die Methode sortiert eine Reihung von Punkteständen absteigend.

a) Erstelle eine Tracetabelle für r = [30, 80, 50, 90, 60] mit den Spalten i | Vergleiche | Vertauschungen | Reihung nach dem Durchlauf (eine Zeile pro Durchlauf der äußeren Schleife). Als Vergleich zählt jede Auswertung von r[j − 1] < r[j].

b) Gib die Gesamtzahl der Vergleiche an und vergleiche sie mit Selectionsort (10 Vergleiche bei 5 Elementen).

c) Gib eine Belegung mit 5 Elementen an, bei der Insertionsort nur 4 Vergleiche braucht.

```
public static void sortiere(int[] r) {
    for (int i = 1; i < r.length; i++) {
        int j = i;
        while (j > 0 && r[j - 1] < r[j]) {
            int hilf = r[j];
            r[j] = r[j - 1];
            r[j - 1] = hilf;
            j--;
        }
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-06

WARM-UP W2-07 Lückencode

Welt 2 – Rangliste · ⏱ 5 min

Die Innenschleife

Die Methode insertionSort der Klasse Rangliste sortiert die Einträge absteigend nach Punkten. Vorgegeben sind besser(a, b) (liefert a.getPunkte() > b.getPunkte() und zählt den Vergleich) und vertausche(i, j).

a) Ergänze die Lücken (1) bis (4).

b) Begründe, warum die Reihenfolge der beiden Teilbedingungen in der while-Bedingung wichtig ist.

c) Erläutere, warum zwei Einträge mit gleicher Punktzahl ihre Reihenfolge behalten (Stabilität).

```
public void insertionSort() {
    for (int i = 1; i < anzahl; i++) {
        int j = ___(1)___;
        while (___(2)___ && besser(eintraege[j], eintraege[___(3)___])) {
            vertausche(j, j - 1);
            ___(4)___;
        }
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-07

WARM-UP W2-08 Tracetabelle

Welt 2 – Rangliste · ⏱ 6 min

Bubblesort mit Notbremse

Die Methode sortiert eine Reihung von Punkteständen absteigend und bricht ab, sobald ein Durchlauf ohne Vertauschung bleibt.

a) Erstelle eine Tracetabelle für r = [90, 50, 70, 80, 20] mit den Spalten Durchlauf | Vertauschungen | Reihung nach dem Durchlauf.

b) Gib die Gesamtzahl der Vergleiche r[j] < r[j + 1] an.

c) Gib für 5 Elemente die Vergleichszahl im besten und im schlechtesten Fall an und nenne jeweils eine passende Belegung.

```
public static void sortiere(int[] r) {
    boolean getauscht = true;
    int durchlauf = 0;
    while (getauscht) {
        getauscht = false;
        for (int j = 0; j < r.length - 1 - durchlauf; j++) {
            if (r[j] < r[j + 1]) {
                int hilf = r[j];
                r[j] = r[j + 1];
                r[j + 1] = hilf;
                getauscht = true;
            }
        }
        durchlauf++;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-08

WARM-UP W2-09 Methode implementieren

Welt 2 – Rangliste · ⏱ 8 min

Neuer Highscore!

Implementiere in der Klasse Rangliste die Methode einfuegenSortiert(e: Eintrag): Wahrheitswert.

- Die Liste ist absteigend nach Punkten sortiert und soll es bleiben.
- Liefert false, wenn e null ist oder die Liste voll ist (anzahl = Länge von eintraege); sonst true.
- Schlechtere Einträge rücken von hinten her eine Position nach rechts (ohne vertausche). e kommt HINTER alle gleich guten Einträge.
- Vergleiche nur über besser(a, b) (liefert a.getPunkte() > b.getPunkte()).

Beispiel: 1500, 1200, 900, 700 → einfuegenSortiert(Dana 1000) → 1500, 1200, Dana 1000, 900, 700

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-09

WARM-UP W2-10 Verändern und Erweitern

Welt 2 – Rangliste · ⏱ 6 min

Andersherum und mitgezählt

Die Methode sortiert eine Reihung absteigend (Bubblesort ohne Abbruch).

a) Verändere die Methode so, dass sie aufsteigend sortiert.

b) Erweitere die Methode so, dass sie die Anzahl der Vertauschungen zurückgibt (Rückgabetyp Ganzzahl).

c) Gib den Rückgabewert der veränderten Methode für r = [4, 3, 2, 1] an und begründe, warum es keine Belegung mit 4 Elementen gibt, die mehr Vertauschungen erzeugt.

```
public static void sortiere(int[] r) {
    for (int d = 0; d < r.length - 1; d++) {
        for (int j = 0; j < r.length - 1 - d; j++) {
            if (r[j] < r[j + 1]) {
                int hilf = r[j];
                r[j] = r[j + 1];
                r[j + 1] = hilf;
            }
        }
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-10

WARM-UP W2-11 Code ↔ Struktogramm

Welt 2 – Rangliste · ⏱ 6 min

Halbieren statt Durchsuchen

Übersetze die Java-Methode in ein Struktogramm nach Abi-Notation:

Kopf mit Signatur (deutsche Typnamen), Zuweisung mit ←, solange-Schleife, geschachtelte Verzweigung, „gib … zurück“.

Gib außerdem an, welche Voraussetzung r erfüllen muss, damit die Methode korrekt arbeitet.

```
public static int binaer(int[] r, int x) {
    int lo = 0;
    int hi = r.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (r[mid] == x) {
            return mid;
        } else if (r[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return -1;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-11

WARM-UP W2-12 Fehler im Struktogramm

Welt 2 – Rangliste · ⏱ 6 min

Die wackelige Blase

Das Struktogramm soll die Reihung r absteigend sortieren (Bubblesort). Es enthält drei Fehler.

a) Finde die drei Fehler und erläutere jeweils ihre Auswirkung.

b) Gib ein korrigiertes Struktogramm an.

```
blasen(r: Reihung von Ganzzahl): nichts
  für d von 0 bis länge(r) − 2
    für j von 0 bis länge(r) − 1 − d
      falls r[j] > r[j + 1]
        dann: r[j] ← r[j + 1]
              r[j + 1] ← r[j]
        sonst: –
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-12

WARM-UP W2-13 Funktionsweise erläutern

Welt 2 – Rangliste · ⏱ 7 min

Wie lange dauert das Ranking?

Der Spiele-Server sortiert die Punktestände mit der Methode sortiere.

a) Gib an, um welches Sortierverfahren es sich handelt, und erläutere die Funktionsweise. In welcher Reihenfolge wird sortiert?

b) Bestimme die Anzahl der Vergleiche r[j] < r[m] für 5 Elemente und für n Elemente. Hängt die Zahl von der Belegung ab?

c) Für 10 000 Einträge braucht der Server 0,3 s. Schätze die Zeit für 20 000 und 40 000 Einträge ab und begründe.

```
public static void sortiere(int[] r) {
    for (int ende = r.length - 1; ende > 0; ende--) {
        int m = 0;
        for (int j = 1; j <= ende; j++) {
            if (r[j] < r[m]) {
                m = j;
            }
        }
        int hilf = r[m];
        r[m] = r[ende];
        r[ende] = hilf;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-13

WARM-UP W2-14 Tracetabelle

Welt 2 – Rangliste · ⏱ 7 min

Die Halbierungs-Jagd

Gegeben ist die aufsteigend sortierte Reihung r = [3, 8, 12, 17, 21, 25, 30, 34, 41, 47, 52] (Indizes 0 bis 10).

a) Erstelle je eine Tracetabelle mit den Spalten lo | hi | mid | r[mid] (eine Zeile pro Schleifendurchlauf) für binaer(r, 47) und binaer(r, 50). Gib jeweils den Rückgabewert an.

b) Gib an, wie viele Schleifendurchläufe bei n = 11 höchstens nötig sind.

c) Begründe, dass bei n = 100 höchstens 7 Durchläufe nötig sind, und gib an, wie viele es bei n = 200 höchstens sind.

```
binaer(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl
  lo ← 0
  hi ← länge(r) − 1
  solange lo ≤ hi
    mid ← (lo + hi) / 2     // ganzzahlige Division
    falls r[mid] = x
      dann: gib mid zurück
      sonst: falls r[mid] < x
               dann: lo ← mid + 1
               sonst: hi ← mid − 1
  gib −1 zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W2-14